CHALMERS
UNIVERSITY OF TECHNOLOGY

UNIVERSITY OF GOTHENBURG

DIT357: Fundamentals of Distributed Systems

Lecture 12: Replication and Consistency

Hans-Martin Heyn,
Universitetslektor

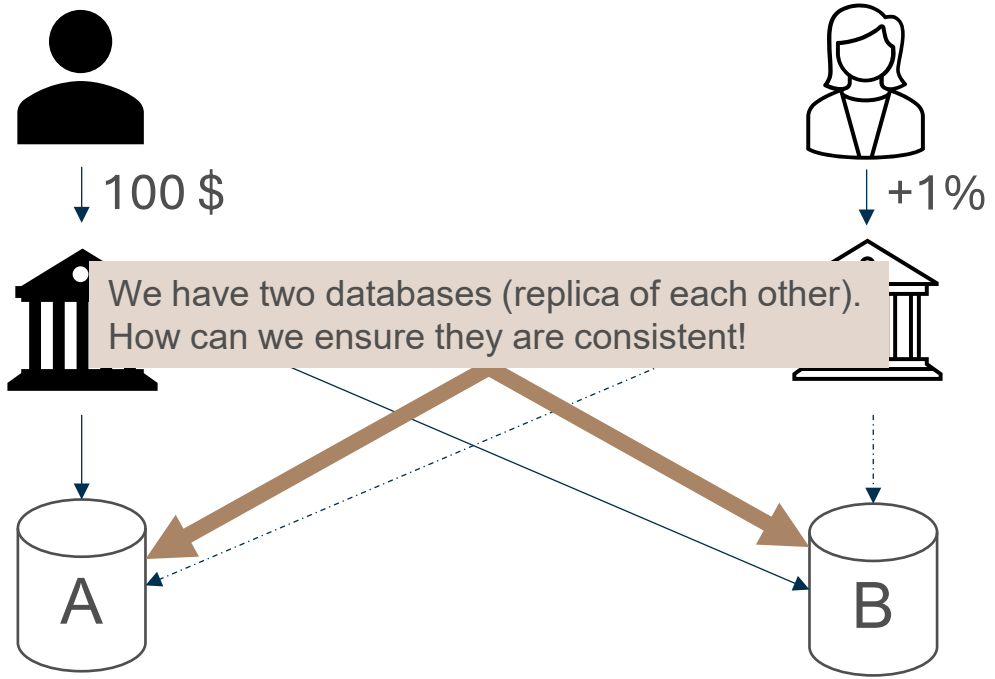
hans-martin.heyn@gu.se

Remember?

A customer P_1 has 1000 SEK on her account. She deposits in Luleå another 100 SEK.

Exactly at the same time, banker P_2 in Stockholm authorises a 1% special interest increase.

The bank keeps two replicated databases.



Classroom Discussion

Why did the bank have multiple databases?

When are replicas “consistent”?

Why did the bank have multiple databases?

- Improved performance:
  - By spreading the requests between different replicated parts quick responses can be achieved by placing copies close to clients.
- Enhanced scalability:
  - The load can be distributed over a set of replicated servers.
- Increased reliability:
  - If one copy fails, we still have a replicated second copy to work with.
  - Protection against malicious attacks: Tempered copies can be identified.

The problem™:

Having multiple copies, means that when any copy changes, that change should be made at all copies: replicas need to be kept the same, that is, be kept consistent.

Conflicting operations

- **Read–write conflict:** a read operation and a write operation act concurrently (on the same element)
- **Write–write conflict:** two concurrent write operations (on the same element)

Main issue

- To keep replicas consistent, we generally need to ensure that all conflicting operations are done **in the same order everywhere**.
- Guaranteeing global ordering on conflicting operations may be a costly operation, downgrading scalability.

Path to solution

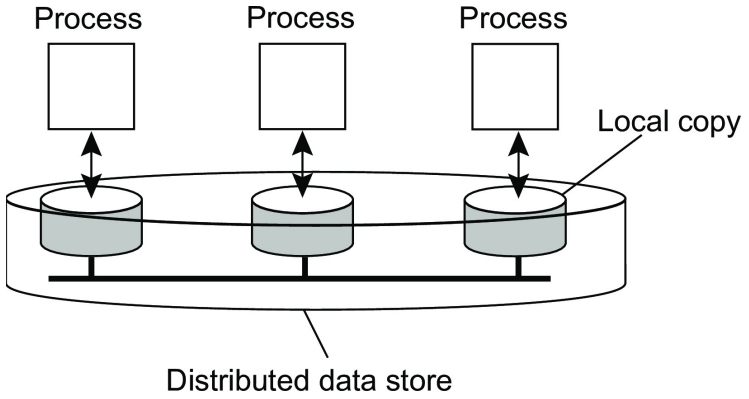
Weaken consistency requirements so that (hopefully) global synchronisation can be avoided

The role of consistency models

Definition of consistency model

A **contract between a (distributed) data store and processes**, in which the data store specifies precisely what the results of read and write operations are in the presence of concurrency and the absence of a global clock.

A data store is a distributed collection of storages

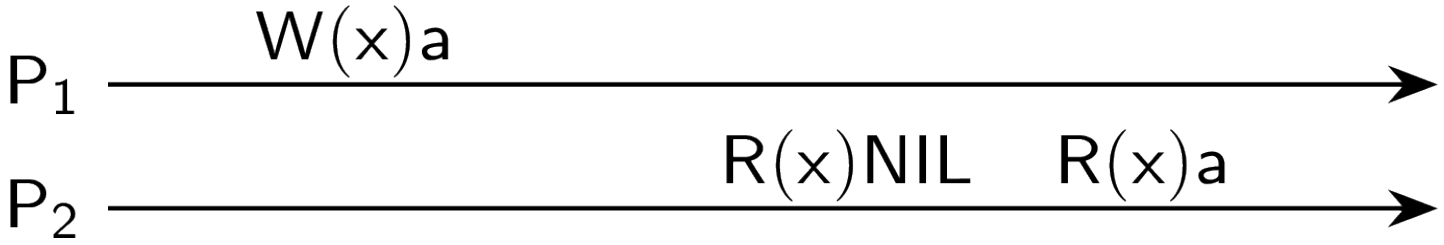


Data-centric consistency

Notation

Read and write operations

- $W_i(x)a$: Process P_i writes value a to x
- $R_i(x)b$: Process P_i reads value b from x
- All data items initially have value NIL

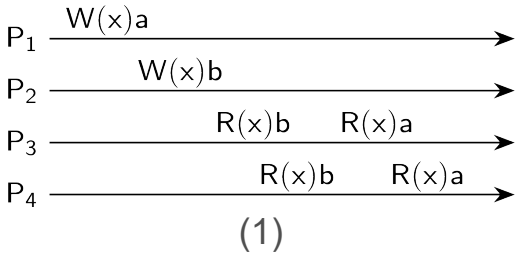


Sequential and Causal Consistency

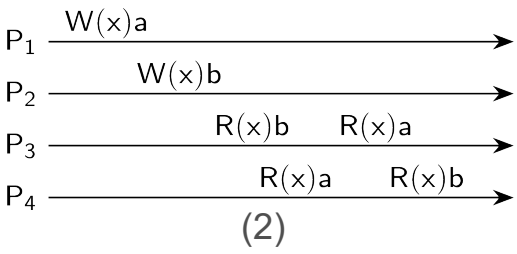
Sequential consistency

Definition of sequential consistency

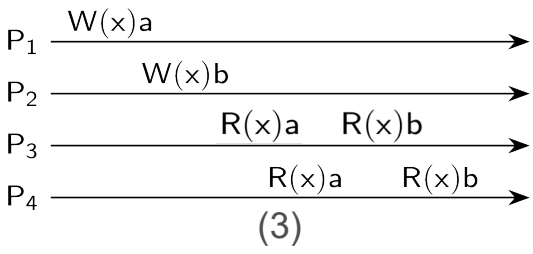
The result of any execution is the same as if the **operations by all processes** on the data store were **executed in some sequential order** and the **operations of each individual process appear in this sequence** in the order specified by its program.



(1)



(2)

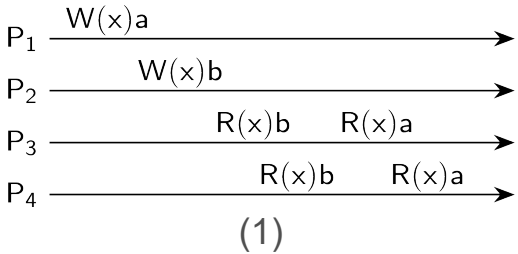


(3)

Sequential consistency

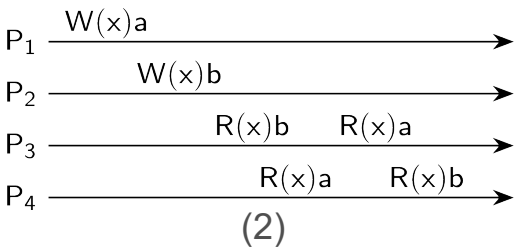
Definition of sequential consistency

The result of any execution is the same as if the **operations by all processes** on the data store were **executed in some sequential order** and the **operations of each individual process appear in this sequence** in the order specified by its program.



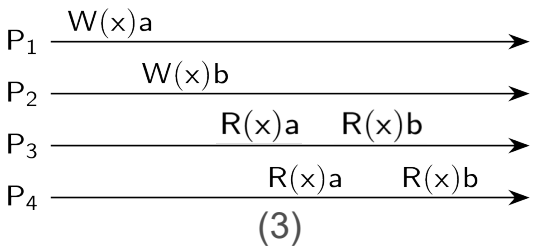
(1)

Sequentially consistent



(2)

Not sequentially consistent



(3)

Sequentially consistent

Totally ordered

Sequential consistency

Process P_1	Process P_2	Process P_3
x ← 1;	y ← 1;	z ← 1;
print(y,z);	print(x,z);	print(x,y);

How many possible combinations of execution exist?

- $n_{tot} = 6! = 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 720(!)$ different combinations!

But remember, that the processes execute in a specific order:

- $n_{tot,valid} = \frac{6!}{2^3} = 90$ valid combination. That is still a lot.

Example execution sequences

Process P_1	Process P_2	Process P_3
x ← 1;	y ← 1;	z ← 1;
print(y,z);	print(x,z);	print(x,y);

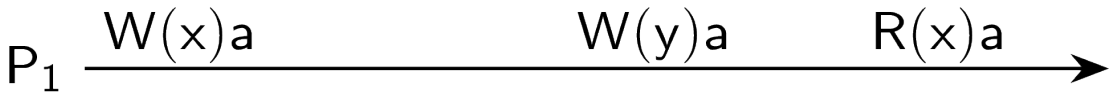
Execution 1	Execution 2	Execution 3	Execution 4
P_1: x ← 1;	P_1: x ← 1;	P_2: y ← 1;	P_2: y ← 1;
P_1: print(y,z);	P_2: y ← 1;	P_3: z ← 1;	P_1: x ← 1;
P_2: y ← 1;	P_2: print(x,z);	P_3: print(x,y);	P_3: z ← 1;
P_2: print(x,z);	P_1: print(y,z);	P_2: print(x,z);	P_2: print(x,z);
P_3: z ← 1;	P_3: z ← 1;	P_1: x ← 1;	P_1: print(y,z);
P_3: print(x,y);	P_3: print(x,y);	P_1: print(y,z);	P_3: print(x,y);

Example execution sequences

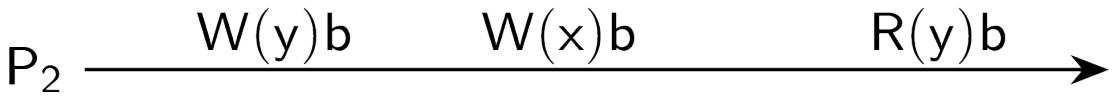
Process P_1	Process P_2	Process P_3
x ← 1;	y ← 1;	z ← 1;
print(y,z);	print(x,z);	print(x,y);

Execution 1	Execution 2	Execution 3	Execution 4
P_1: x ← 1;	P_1: x ← 1;	P_2: y ← 1;	P_2: y ← 1;
P_1: print(y,z);	P_2: y ← 1;	P_3: z ← 1;	P_1: x ← 1;
P_2: y ← 1;	P_2: print(x,z);	P_3: print(x,y);	P_3: z ← 1;
P_2: print(x,z);	P_1: print(y,z);	P_2: print(x,z);	P_2: print(x,z);
P_3: z ← 1;	P_3: z ← 1;	P_1: x ← 1;	P_1: print(y,z);
P_3: print(x,y);	P_3: print(x,y);	P_1: print(y,z);	P_3: print(x,y);
Prints: 001011	*Prints:* 101011	*Prints:* 010111	*Prints:* 111111

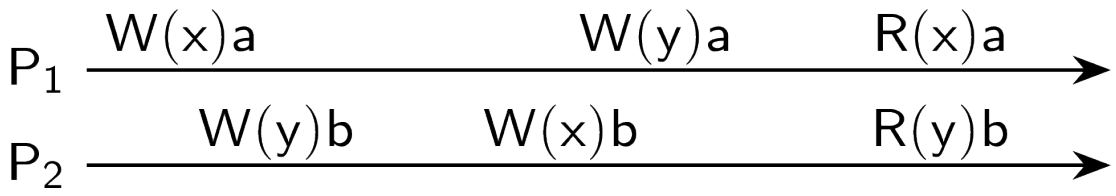
Example 2 execution sequences



Example 2 execution sequences



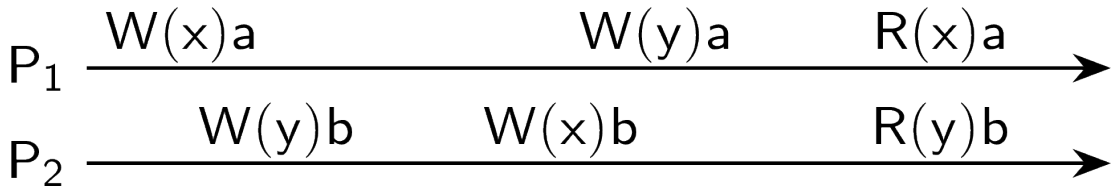
Example 2 execution sequences



Remember:

- P_1 and P_2 act concurrently.
- We do not have a global clock!

Example 2 execution sequences

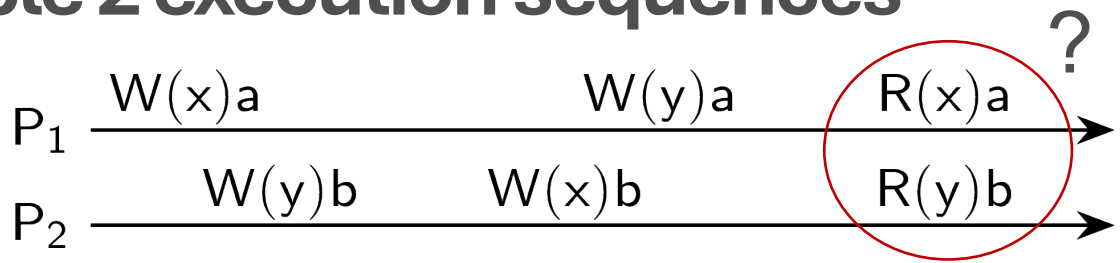


Remember:

- P_1 and P_2 act concurrently.
- We do not have a global clock!

Possible ordering of operations	Result	
$W_1(x)a;\ W_1(y)a;\ W_2(y)b;\ W_2(x)b$	$R_1(x)b$	$R_2(y)b$
$W_1(x)a;\ W_2(y)b;\ W_1(y)a;\ W_2(x)b$	$R_1(x)b$	$R_2(y)a$
$W_1(x)a;\ W_2(y)b;\ W_2(x)b;\ W_1(y)a$	$R_1(x)b$	$R_2(y)a$
$W_2(y)b;\ W_1(x)a;\ W_1(y)a;\ W_2(x)b$	$R_1(x)b$	$R_2(y)a$
$W_2(y)b;\ W_1(x)a;\ W_2(x)b;\ W_1(y)a$	$R_1(x)b$	$R_2(y)a$
$W_2(y)b;\ W_2(x)b;\ W_1(x)a;\ W_1(y)a$	$R_1(x)a$	$R_2(y)a$

Example 2 execution sequences



Remember:

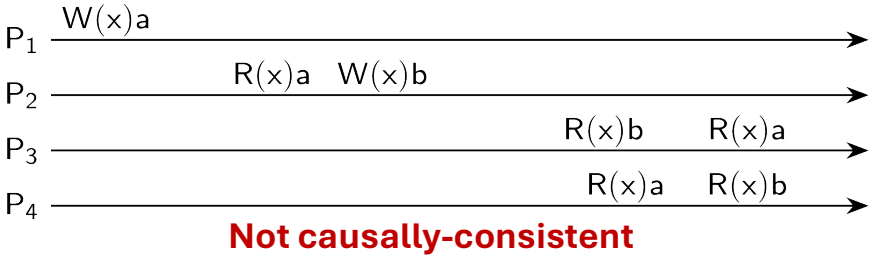
- P_1 and P_2 act concurrently.
- We do not have a global clock!

Sequential consistency is not (necessarily) compositional!

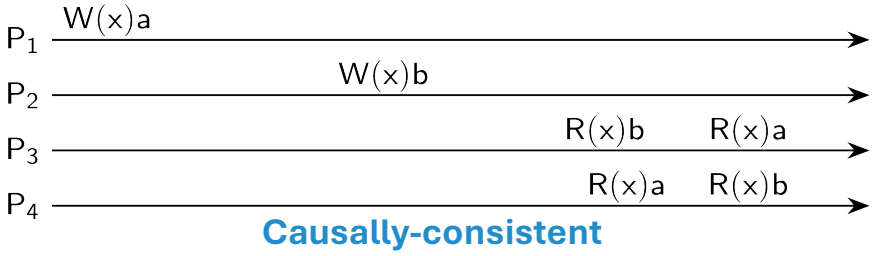
Possible ordering of operations	Result	
$W_1(x)a;\ W_1(y)a;\ W_2(y)b;\ W_2(x)b$	$R_1(x)b$	$R_2(y)b$
$W_1(x)a;\ W_2(y)b;\ W_1(y)a;\ W_2(x)b$	$R_1(x)b$	$R_2(y)a$
$W_1(x)a;\ W_2(y)b;\ W_2(x)b;\ W_1(y)a$	$R_1(x)b$	$R_2(y)a$
$W_2(y)b;\ W_1(x)a;\ W_1(y)a;\ W_2(x)b$	$R_1(x)b$	$R_2(y)a$
$W_2(y)b;\ W_1(x)a;\ W_2(x)b;\ W_1(y)a$	$R_1(x)b$	$R_2(y)a$
$W_2(y)b;\ W_2(x)b;\ W_1(x)a;\ W_1(y)a$	$R_1(x)a$	$R_2(y)a$

Causal consistency

- Sequential consistency is still (too) restrictive, which can cause significant performance (and scalability) problems.
- We can weaken consistency by making a distinction between events that are causally related and those who are independent (truly concurrent).



Not causally-consistent



Causally-consistent

Definition causal consistency

Writes that are potentially causally related must be seen by all processes in the same order. **Operations that are not causally related can be concurrent.**

Grouping Operations

Grouping operations

Entry consistency: Definition

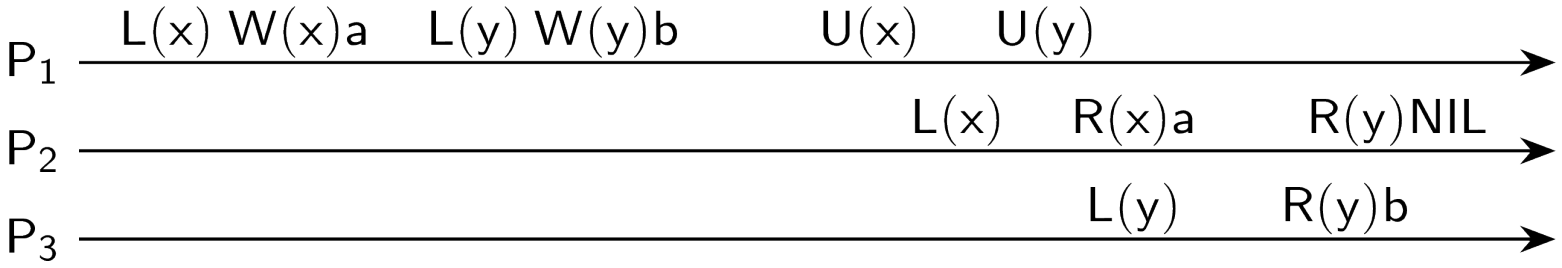
- Accesses to locks are sequentially consistent.
- No access to a lock is allowed to be performed until all previous writes have completed everywhere.
- No data access is allowed to be performed until all previous accesses to locks have been performed.

Basic idea

You don't care that reads and writes of a series of operations are immediately known to other processes. **You just want the effect of the series itself to be known.**

Grouping operations

Entry consistency implies that we need to lock and unlock data.



Exclusive locks

With exclusive access to a lock, one, and only one, process has the right to read and write.

Non-exclusive locks

Several processes can have nonexclusive access to a lock to read only, but not to write.

Grouping operations

Exclusive locks

With exclusive access to a lock, one, and only one, process has the right to read and write.

Non-exclusive locks

Several processes can have nonexclusive access to a lock to read only, but not to write.

Criteria for acquiring a lock:

- Acquiring a lock can succeed only when all updates to its associated shared data have completed.
- Exclusive access to a lock can succeed only if no other process has exclusive or non-exclusive access to that lock.
- Non-exclusive access to a lock is allowed only if any previous exclusive access has been completed, including updates to the lock's associated data.

Grouping operations

Exclusive locks

With exclusive access to a lock, one, and only … has the right to re…

Non-exclusive locks

Several processes can have nonexclusive access to a lock … only, but not to write.

More in the next lecture ☺

Criteria for a…

- Acquiring … iated shared data have…
- Exclusive access to a lock can suc… s has exclusive or non-exclusive access to that lock.
- Non-exclusive access to a lock is allowed only if any previous exclusive access has been completed, including updates to the lock's associated data.

CHALMERS
UNIVERSITY OF TECHNOLOGY

UNIVERSITY OF GOTHENBURG

Client-centric consistency

Eventual consistency

Definition eventual consistency:

Consider a collection of data stores and (concurrent) write operations:

The stores are **eventually consistent when, in lack of updates from a certain moment, all updates to that point are propagated in such a way that replicas will have the same data stored** (until updates are accepted again).

In English: Eventual consistency requires **only that updates are guaranteed to propagate to all replicas** (at some point in the future).

Strong eventual consistency: if there are conflicting updates, have a globally determined resolution mechanism.

Eventual consistency and program consistency

Program consistency:

P is a **monotonic problem** if for any input sets S and T , $P(S) \subseteq P(T)$.

A program solving a monotonic problem can start with incomplete information but is **guaranteed not to have to roll back when missing information becomes available**.

Example: A shopping card

- A typical example of such a problem is filling a (digital) shopping cart: operations can be performed in any order by any server.
- **But what if we also need to remove items?**
- We can maintain monotonicity if we split the add and delete operations into two different sets, and let each of them simply grow to a final state.
- The difference between what has been added and what has been deleted is what needs to be coordinated to reach consistency.

Trade-off in maintaining consistency

Loose consistency

- Easier to implement
- More efficient
- Less consistent

- It is a balance between strictness of consistency and efficiency / ease of implementation.
- How much consistency is needed?
- Banking Systems vs. Social Media
- Good-enough consistency depends on your use case.

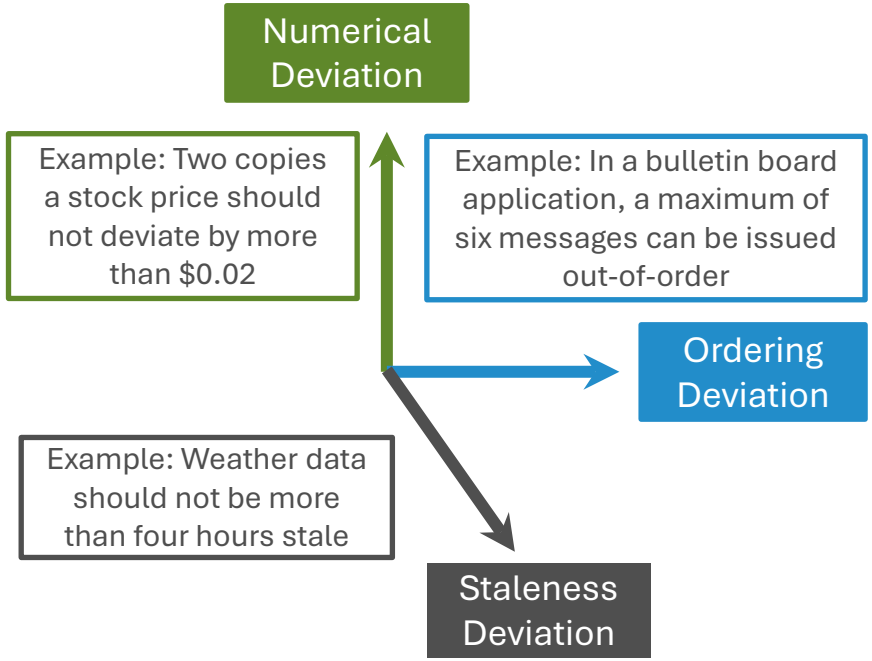
Strict consistency

- Harder to implement
- Less efficient
- More consistent

Continuous consistency

We can differentiate **different dimensions of consistency:**

- replicas may differ in their numerical value
- replicas may differ regarding the order of performed update operations
- replicas may differ in their relative staleness (time since last update)



Consistency unit

Consistency unit (Conit) specifies the data unit over which consistency is measured

Level of consistency is measured along the three dimensions:

- Numerical Deviation
  - For a given replica R, how many updates at other replicas are not yet seen at R? What is the effect of the non-propagated updates on local Conit values?
- Order Deviation
  - For a given replica R, how many local updates are not propagated to other replicas?
- Staleness Deviation
  - For a given replica R, how long has it been since updates were propagated?

Examples: Conit

A fleet owner is interested in knowing how much she pays on average for gas. Whenever a driver tanks gasoline, she reports the amount of gasoline that has been tanked (recorded as g), the price paid (recorded as p), and the total distance since the last time she tanked (recorded by the variable d).

Conit (contains the variables g, p, and d)

- Each replica has a vector clock: ([known] time at A, [known] time at B)
- B sends A operation $[\langle 5, B\rangle : g \leftarrow g + 45]$; A has made this operation permanent (cannot be rolled back).

Replica A

Conit
d = 558 // distance
g = 95 // gas
p = 78 // price

Operation		Result
< 5, B>	g ← g + 45	[g = 45]
< 8, A>	g ← g + 50	[g = 95]
< 9, A>	p ← p + 78	[p = 78]
<10, A>	d ← d + 558	[d = 558]

Vector clock A = (11, 5)
Order deviation = 3
Numerical deviation = (2, 482)

Replica B

Conit
d = 412 // distance
g = 45 // gas
p = 70 // price

Operation		Result
< 5, B>	g ← g + 45	[g = 45]
< 6, B>	p ← p + 70	[p = 70]
< 7, B>	d ← d + 412	[d = 412]

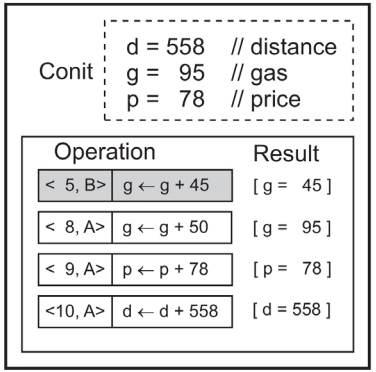
Vector clock B = (0, 8)
Order deviation = 1
Numerical deviation = (3, 686)

Examples: Conit

Conit (contains the variables g, p, and d)

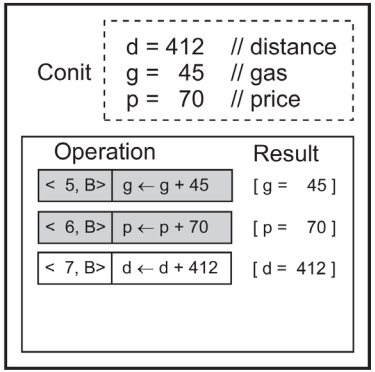
- A has three pending operations ⇒ order deviation = 3
- A missed two operations from B; max diff is 70 + 412 units ⇒ (2, 482)
- Likewise, B is still missing three tentative operations at A worth 95+78+558 = 686 units ⇒ (3, 686)

Replica A



Vector clock A = (11, 5)
Order deviation = 3
Numerical deviation = (2, 482)

Replica B



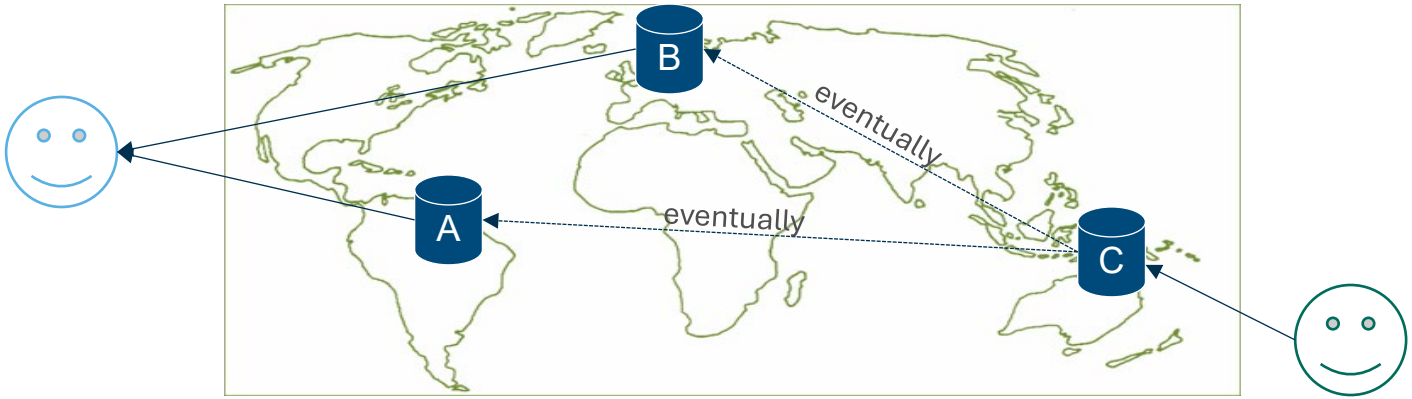
Vector clock B = (0, 8)
Order deviation = 1
Numerical deviation = (3, 686)

- We may restrict order deviation by specifying an acceptable maximal value.
- Requires that a replica knows how much it is deviating from other replicas.
  - Implying that we need separate communication to keep replicas informed.
  - The underlying assumption is that such communication is much less expensive.

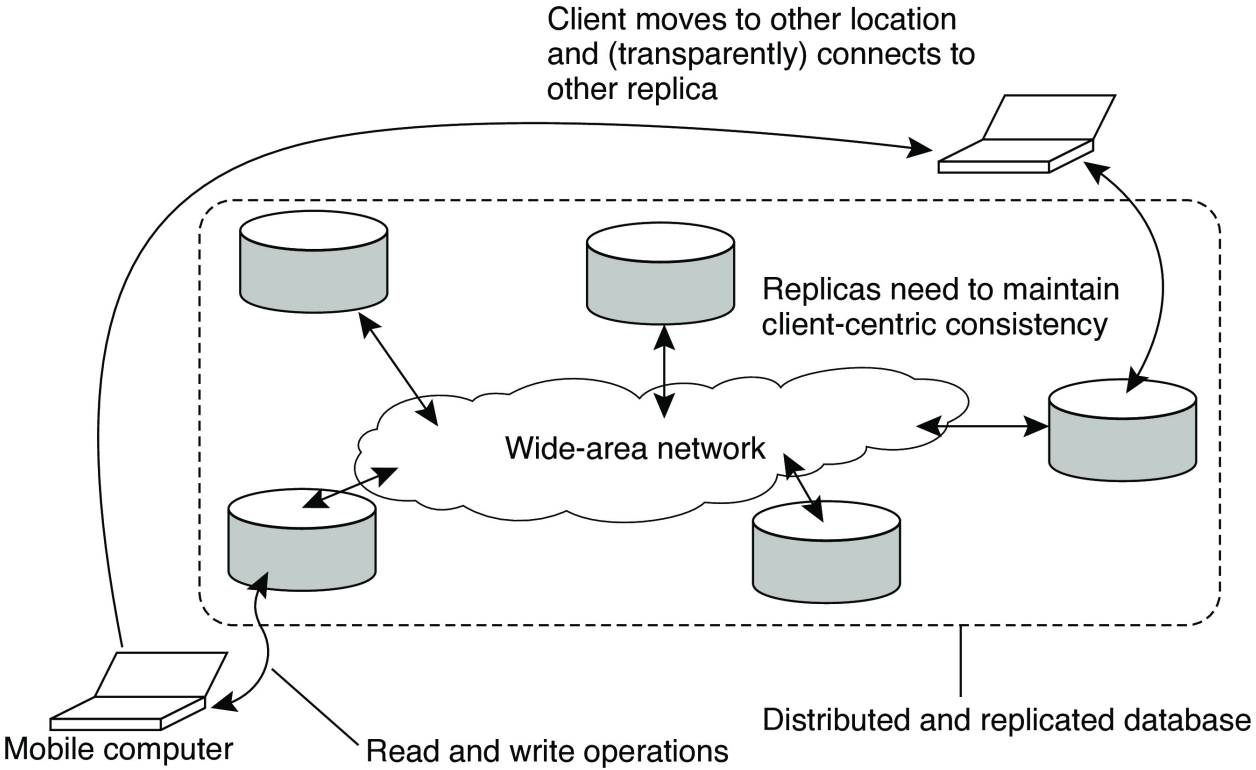
Eventual consistency might not be enough

Eventual Consistency is not good-enough when the client process accesses data from different replicas

- We need consistency guarantees for a single client while accessing the data-store: **client-centric consistency instead of data-centric consistency**



Another example: Data access for mobile users



Another example: Data access for mobile users

Consider a distributed database to which you have access through your notebook. Assume your notebook acts as a front end to the database.

- At location A you access the database doing reads and updates.
- At location B you continue your work, but unless you access the same server as the one at location A, you may detect inconsistencies:
  - your updates at A may not have yet been propagated to B
  - you may be reading newer entries than the ones available at A
  - your updates at B may eventually conflict with those at A

The only thing you really want is that the entries you updated and/or read at A, **are in B the way you left them in A**. In that case, the database will appear to be consistent to you.

Client-centric consistency: Monotonic reads

Monotonic reads examples

Automatically reading your personal calendar updates from different servers. **Monotonic reads guarantees that the user sees all updates, no matter from which server the automatic reading takes place.**

Client-centric consistency: Monotonic reads

Monotonic reads examples

Automatically reading your personal calendar updates from different servers. **Monotonic reads guarantees that the user sees all updates, no matter from which server the automatic reading takes place.**

Reading (not modifying) incoming mail while you are on the move. Each time you connect to a different e-mail server, that server fetches (at least) all the updates from the server you previously visited.

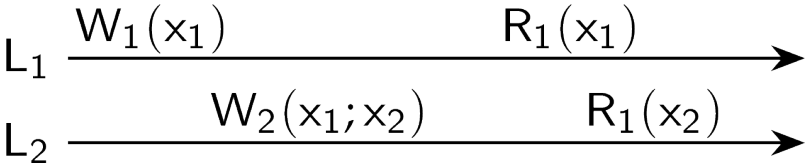
Monotonic reads

Definition monotonic reads:

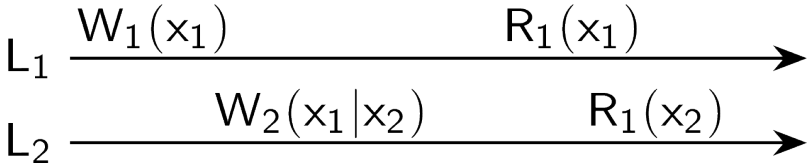
If a process reads the value of a data item x , any successive read operation on x by that process will always return **that same or a more recent value**.

Notations

$W_1(x_2)$ is the write operation by process P_1 that leads to version x_2 of x .
$W_1(x_i\ ;\ x_j)$ indicates P_1 produces version x_j based on a previous version x_i .
$W_1(x_i \mid x_j)$ indicates P_1 produces version x_j concurrently to version x_i .
L_m indicates a local data store.



A monotonic-read consistent data store



A data store that does not provide monotonic reads

Monotonic writes

Monotonic writes examples

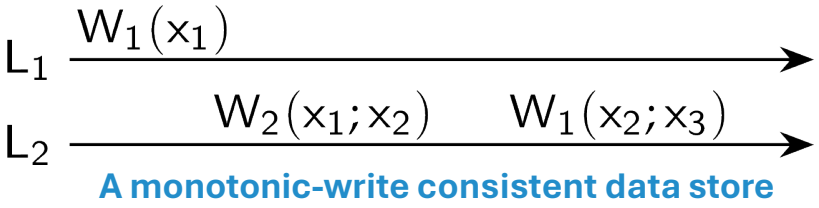
Updating a program at server S_2, and ensuring that all components on which compilation and linking depends, are also placed at S_2.

Maintaining versions of replicated files in the correct order everywhere (propagate the previous version to the server where the newest version is installed).

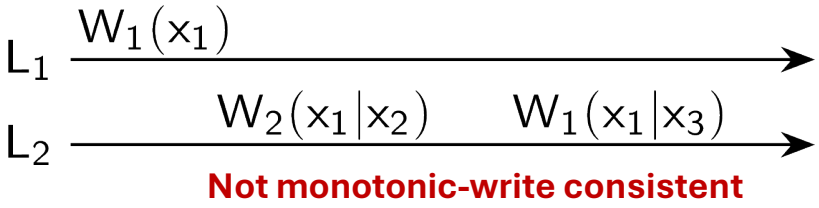
Monotonic writes

Definition read:

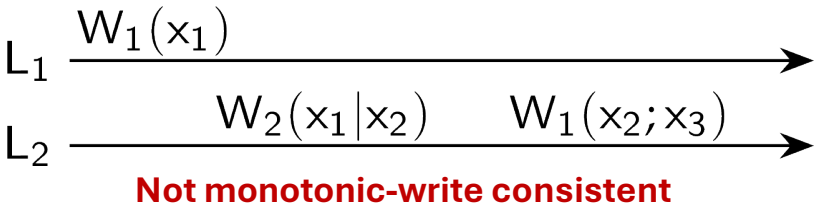
A write operation by a process on a data item x is completed before any successive write operation on x by the same process.



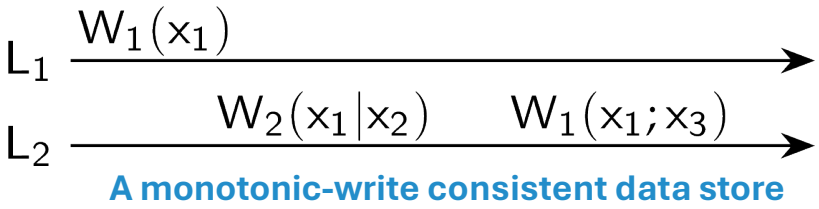
A monotonic-write consistent data store



Not monotonic-write consistent



Not monotonic-write consistent



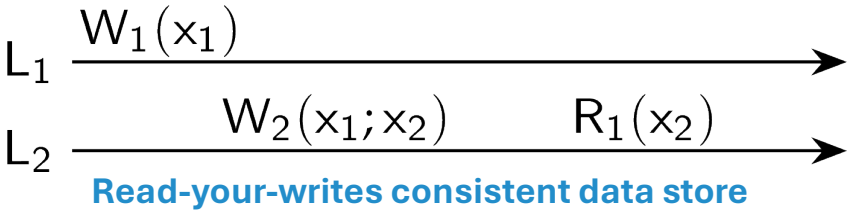
A monotonic-write consistent data store

Read your writes

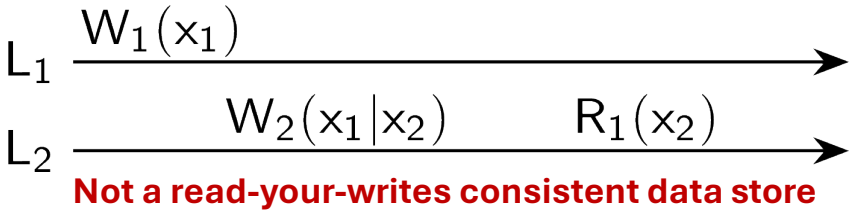
Definition read your writes:

The effect of a write operation by a process on a data item x **will always be seen by a successive read operation** on x by the same process.

In other words, a write operation is always completed before a successive read operation by the same process, no matter the location where that read operation takes place.



Read-your-writes consistent data store

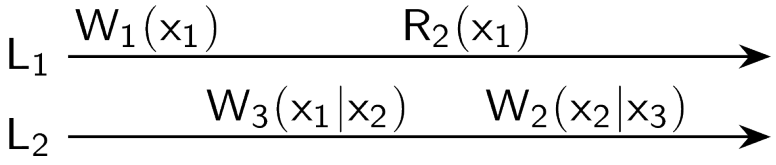


Not a read-your-writes consistent data store

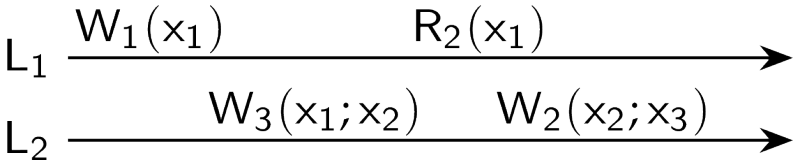
Writes follow reads

Definition writes follow reads

A write operation by a process on a data item x following a previous read operation on x by the same process, is guaranteed to take place **on the same or a more recent value of x that was read**.

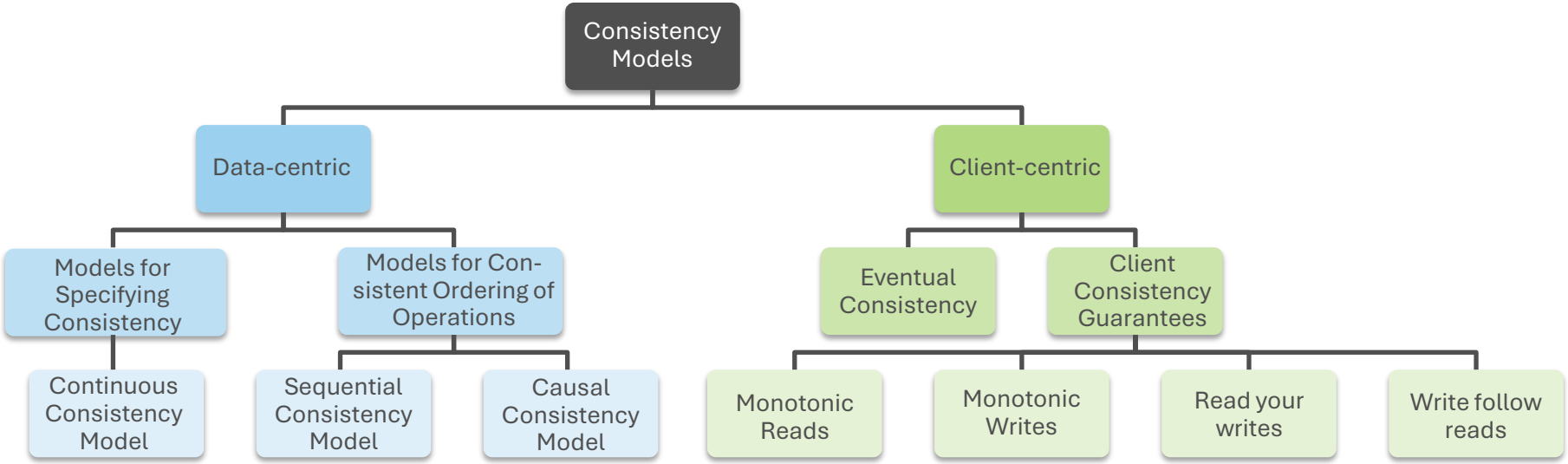


Not a writes-follows-reads consistent data store



Writes-follows-reads consistent data store

Overview of consistency models



Replication Management

Replica Management

Replica management describes where, when, and by whom replicas should be placed.

- Replica(-Server) placement: decides the best locations to place the replica server that can host data-stores.
- Content replication: finds the best server for placing the contents.
- Content distribution: propate (updated) content to the relevant replica servers.

Content replication

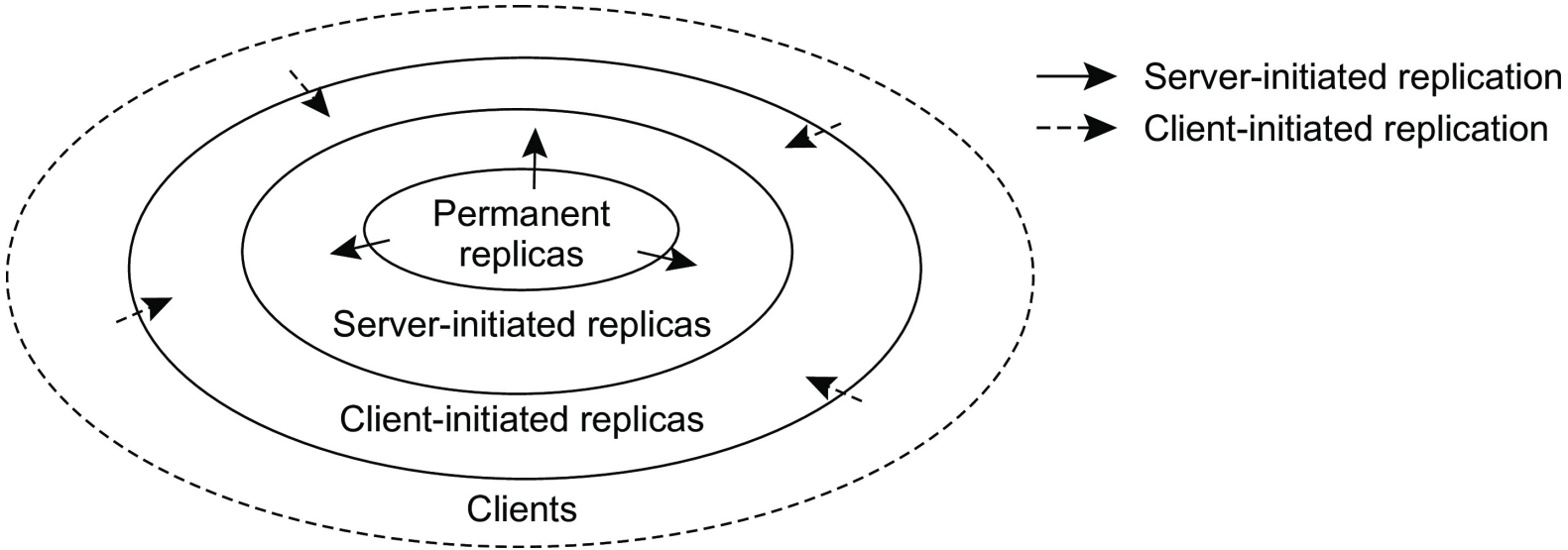
Distinguish different processes

A process is capable of hosting a replica of an object or data:

- Permanent replicas: Process/machine always having a replica.
- Server-initiated replica: Process that can dynamically host a replica on request of another server in the data store.
- Client-initiated replica: Process that can dynamically host a replica on request of a client (**client cache**).

Content replication

This can be modelled as the logical organisation of different kinds of copies of a data store into three concentric rings.



Permanent replicas

Permanent replicas are the initial set of replicas that constitute a distributed data-store. **Typically, small in number.**

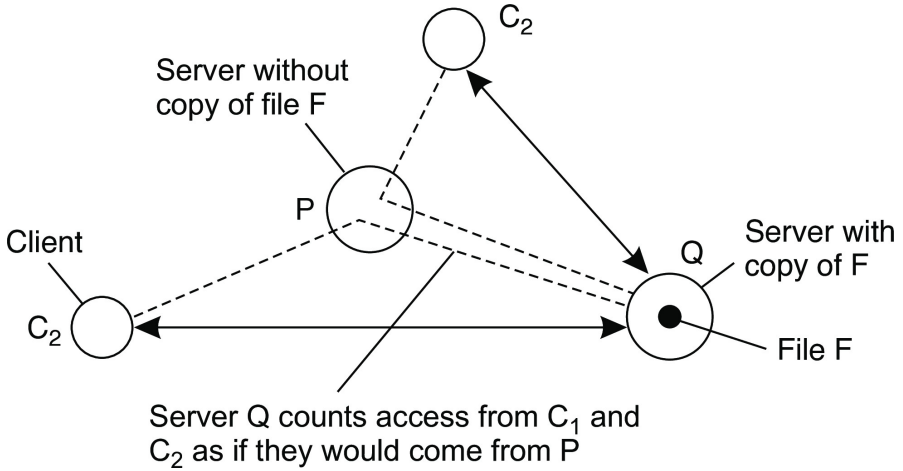
There can be two types of permanent replicas:

- Primary servers
  - One or more servers in an organization
  - Whenever a request arrives, it is forwarded into one of the primary servers
- Mirror sites
  - Geographically spread, and replicas are generally statically configured
  - Clients pick one of the mirror sites to download the data

Server-initiated replicas

Key concept: Counting access requests from different clients

- Keep track of access counts per file, aggregated by considering server closest to requesting clients.
- Number of accesses drops below threshold D ⇒ drop file.
- Number of accesses exceeds threshold R ⇒ replicate file.
- Number of access between D and R ⇒ migrate file.



Client-initiated replicas

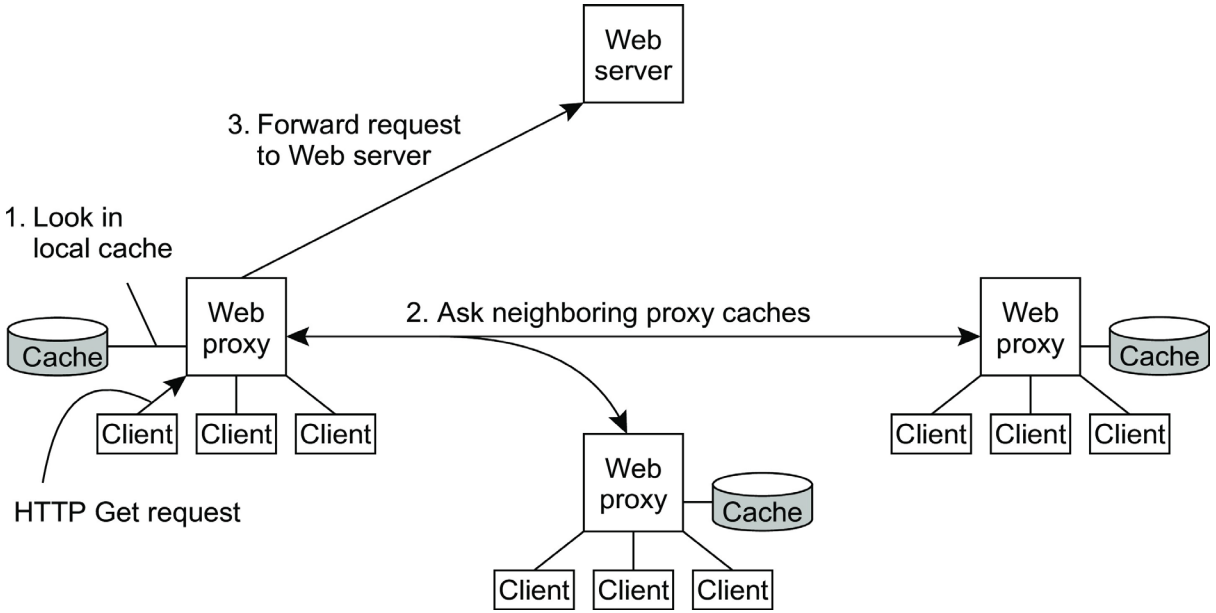
- Client-initiated replicas are known as **client caches**
- Client caches are used only to reduce the access latency of data
  - e.g., Browser caching a web-page locally
- Typically, managing a cache is entirely the responsibility of a client
  - Occasionally, data-store may inform client when the replica has become stale

Replication in the web

Caches at ISPs: Internet Service Providers place caches to

- reduce cross-ISP traffic and
- improve client-side performance.

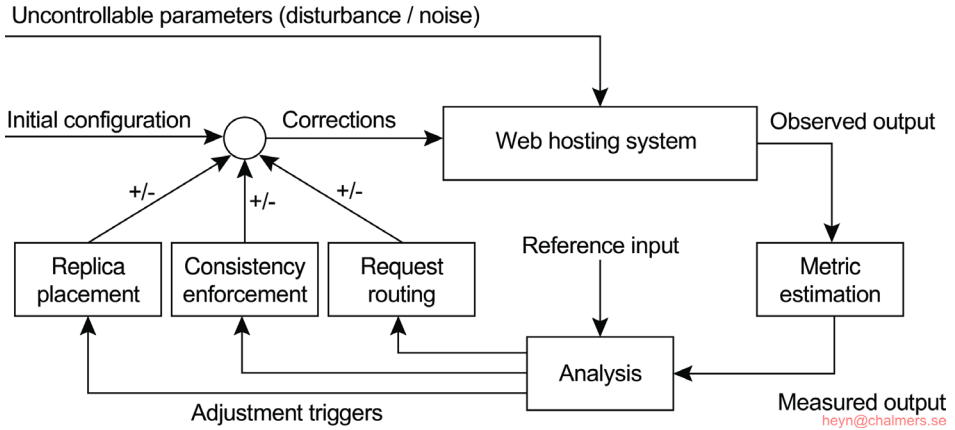
Example: Cooperative web caching:



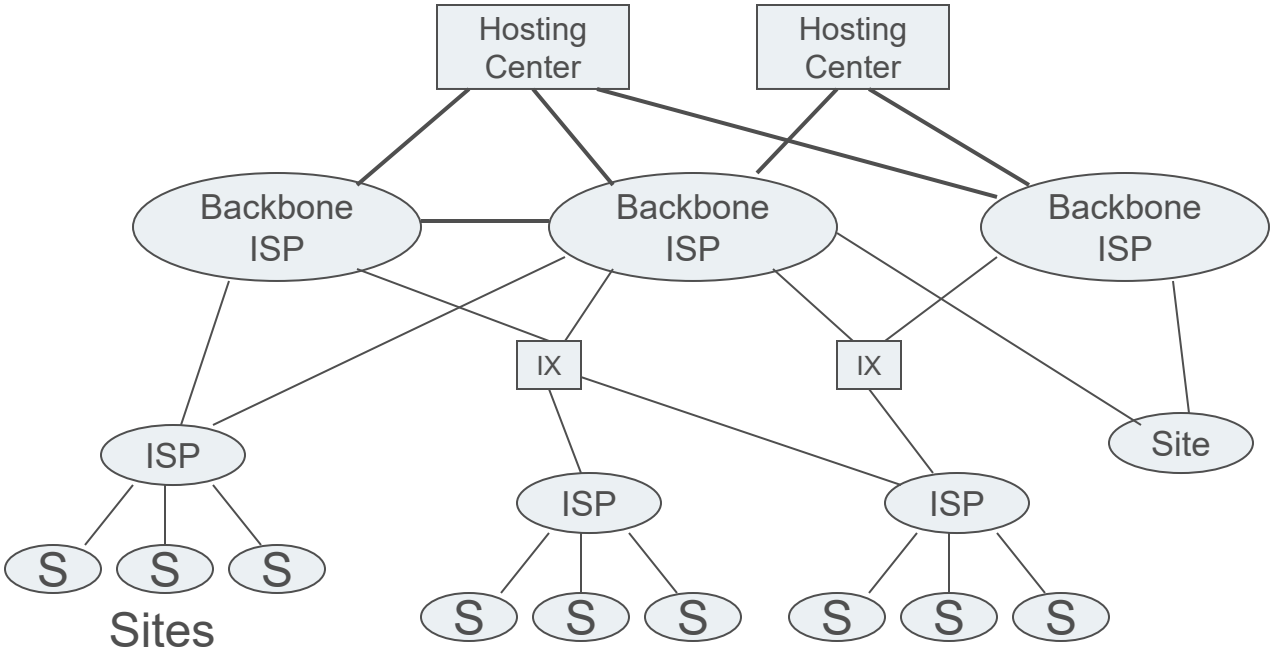
Content Delivery Networks (CDNs)

CDNs provide an infrastructure for distributing and replicating (bandwidth heavy) web content. There are three main tasks:

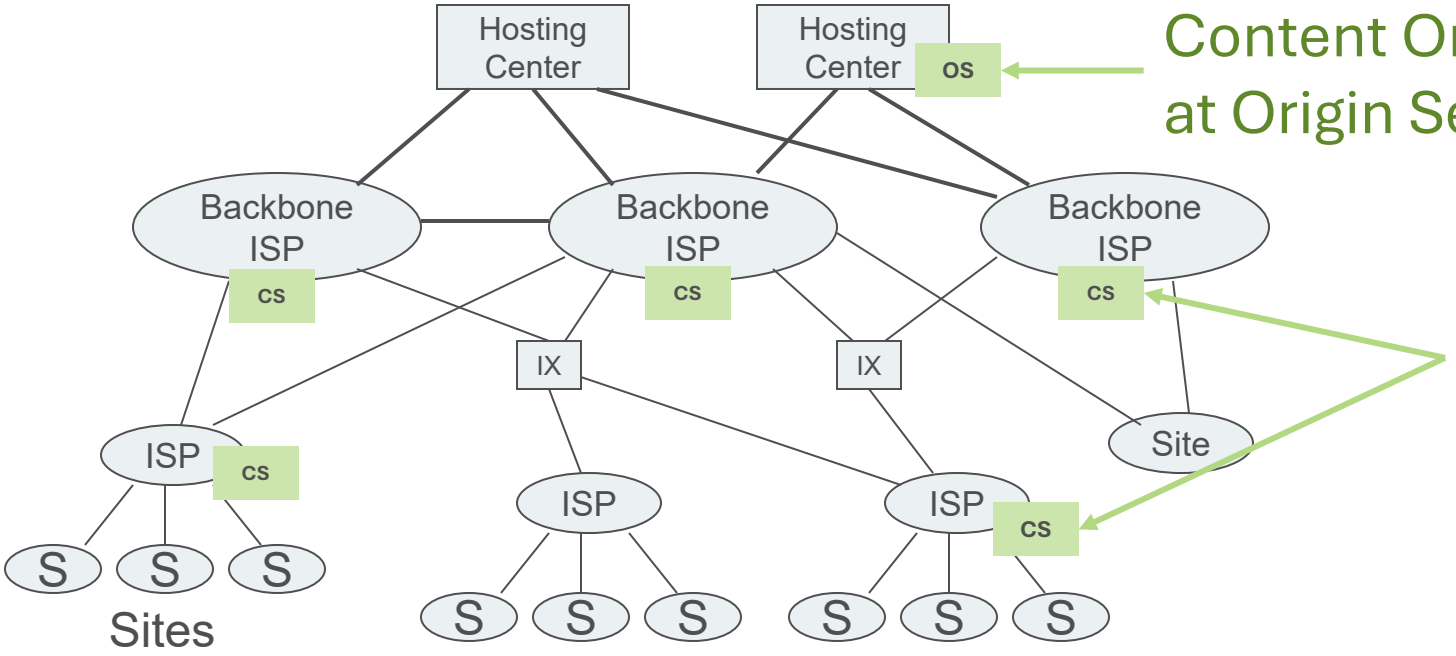
- Metric estimation
- Adaptation triggering
- Taking appropriate measures:
  - Replica placement,
  - Consistency enforcement,
  - Client-request routing.



Content Delivery Networks (CDNs)



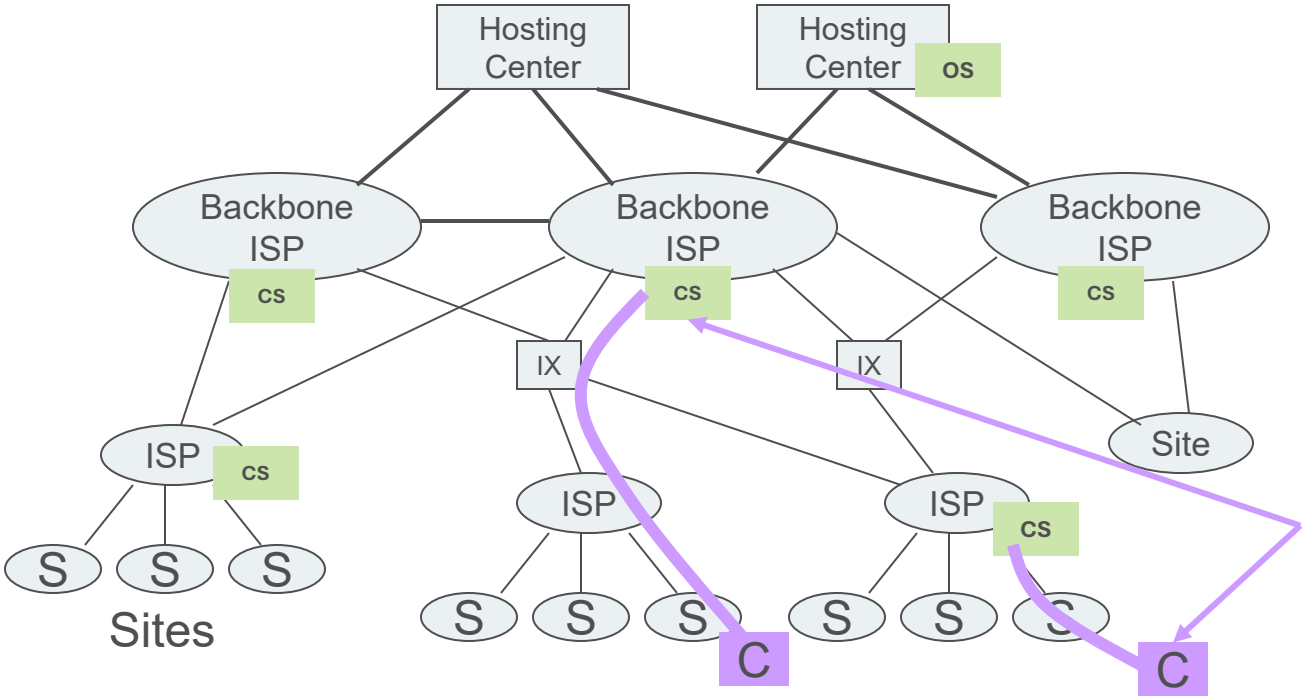
Content Delivery Networks (CDNs)



Content Origin here at Origin Server

Content Servers distributed throughout the Internet

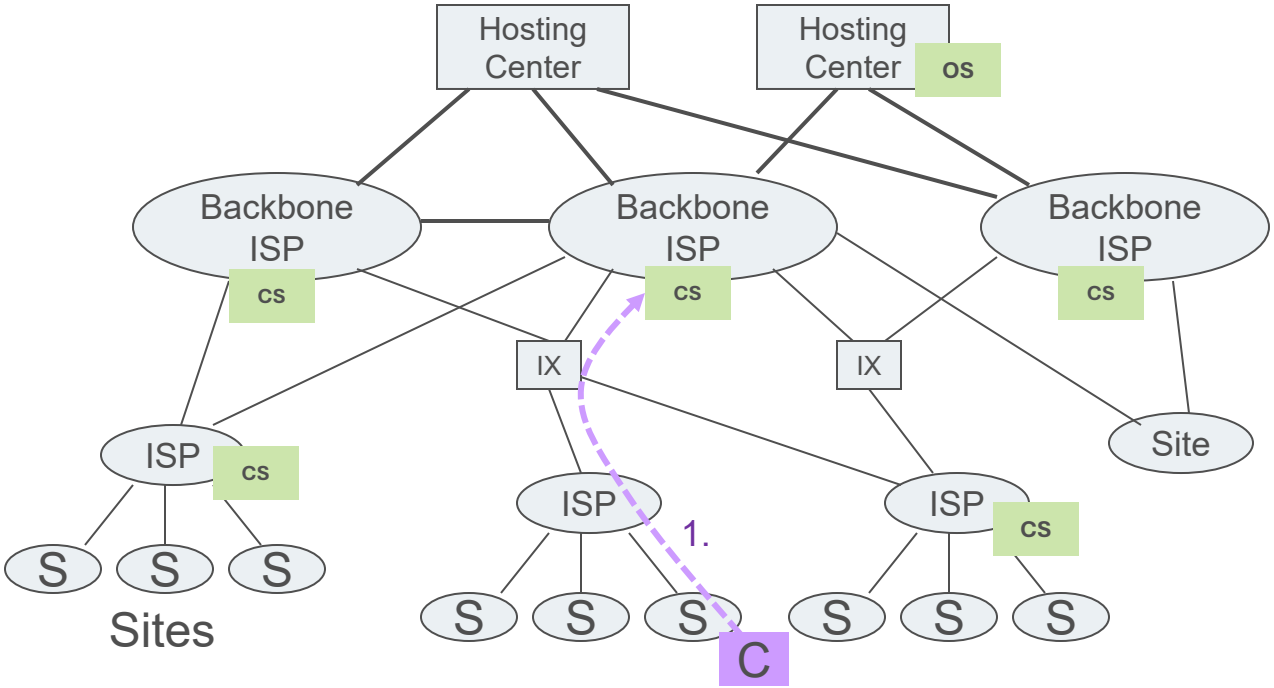
Content Delivery Networks (CDNs)



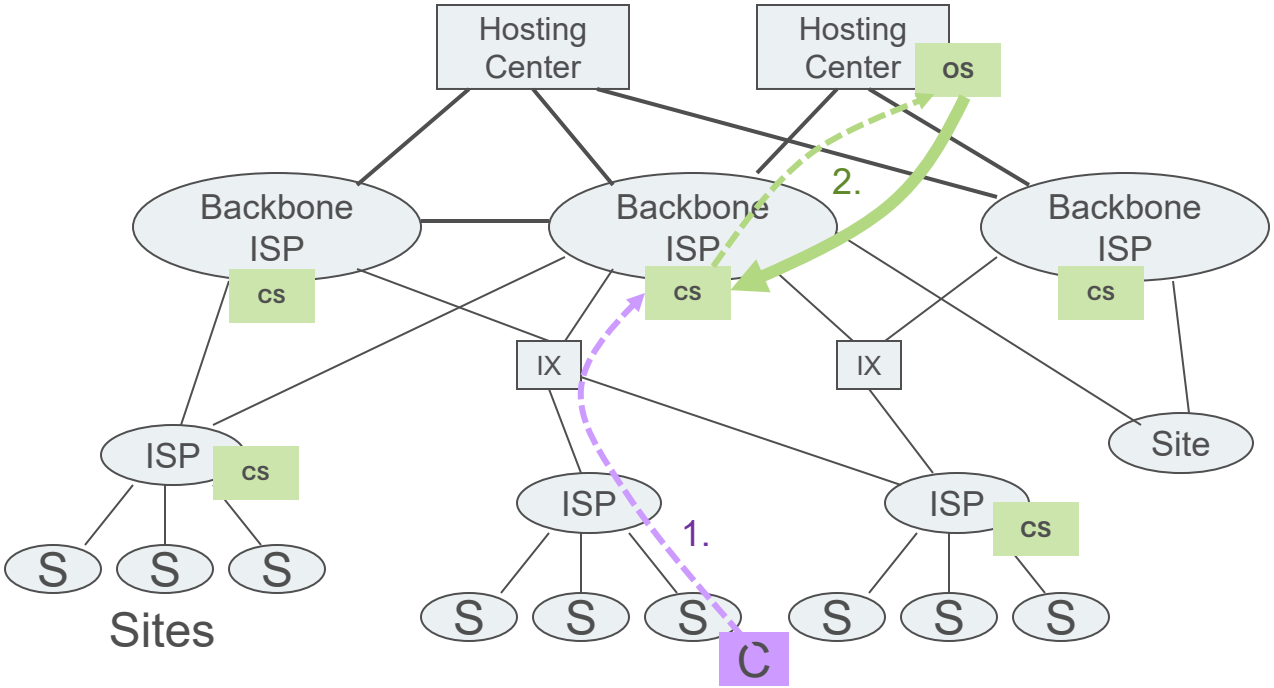
Content is served from content servers nearest to the client

Cached CDN

1. Client requests content.

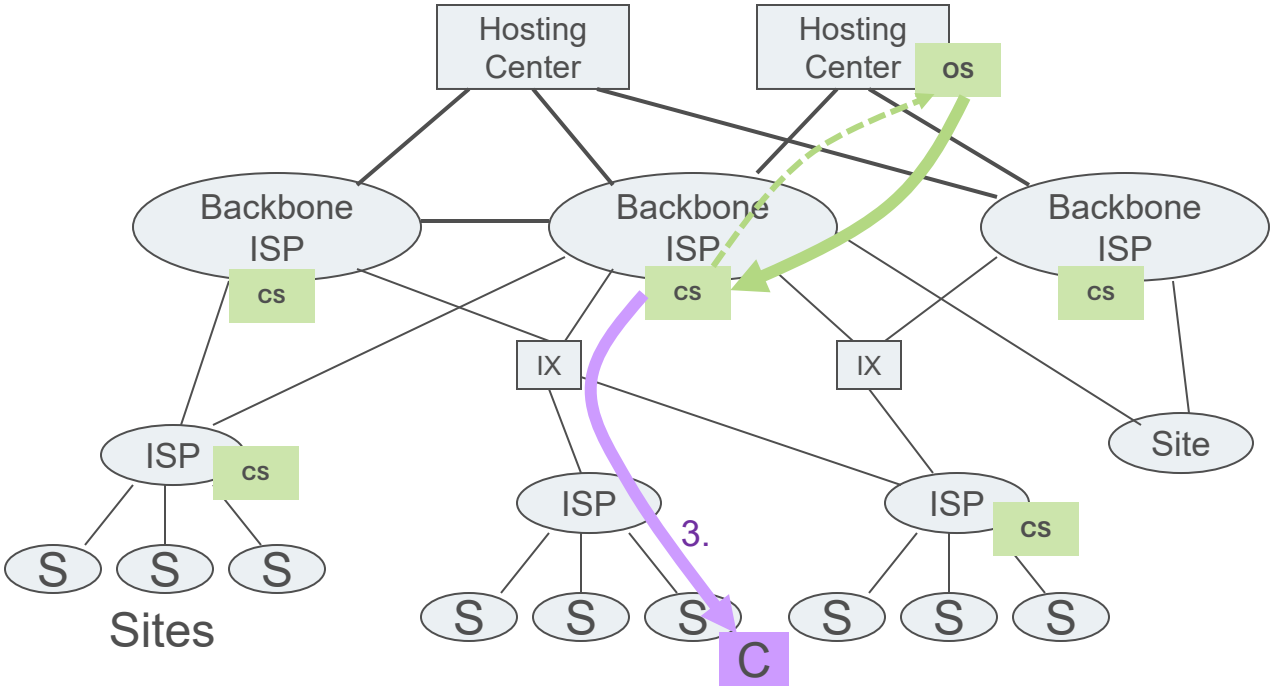


Cached CDN



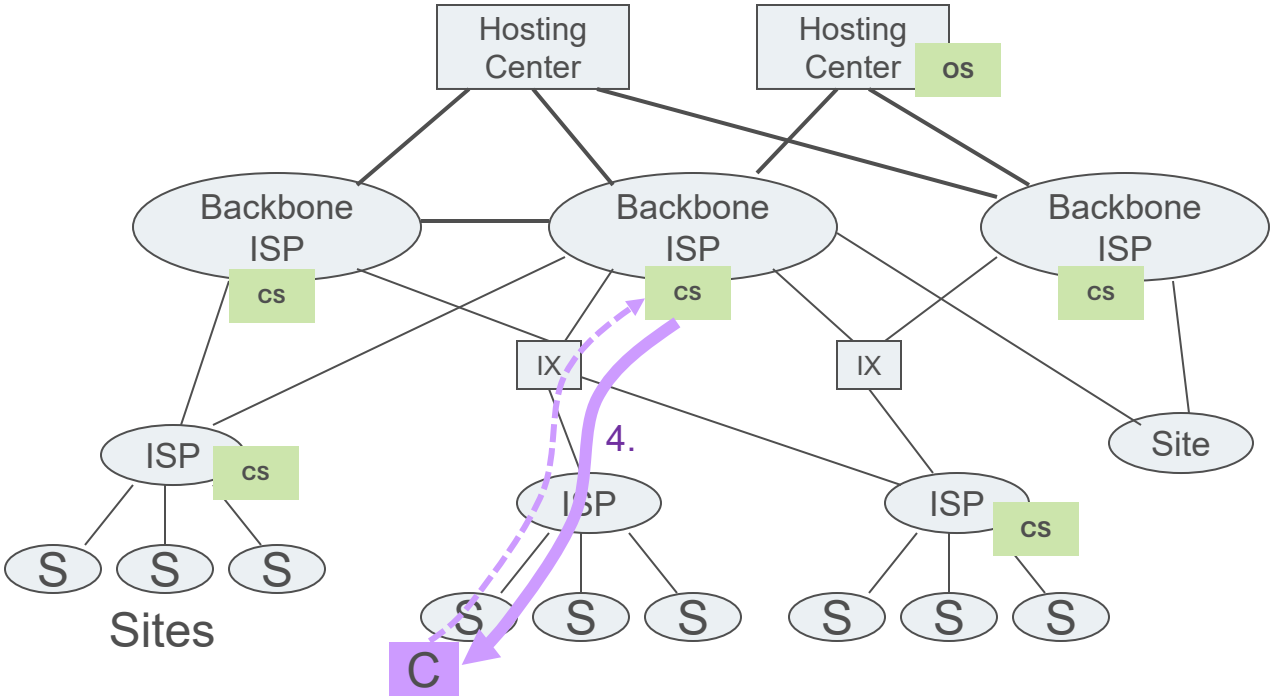
1. Client requests content.
2. CS checks cache, if content is missing it gets it from origin server.

Cached CDN



1. Client requests content.
2. CS checks cache, if content is missing it gets it from origin server.
3. CS caches content, and delivers it to client.

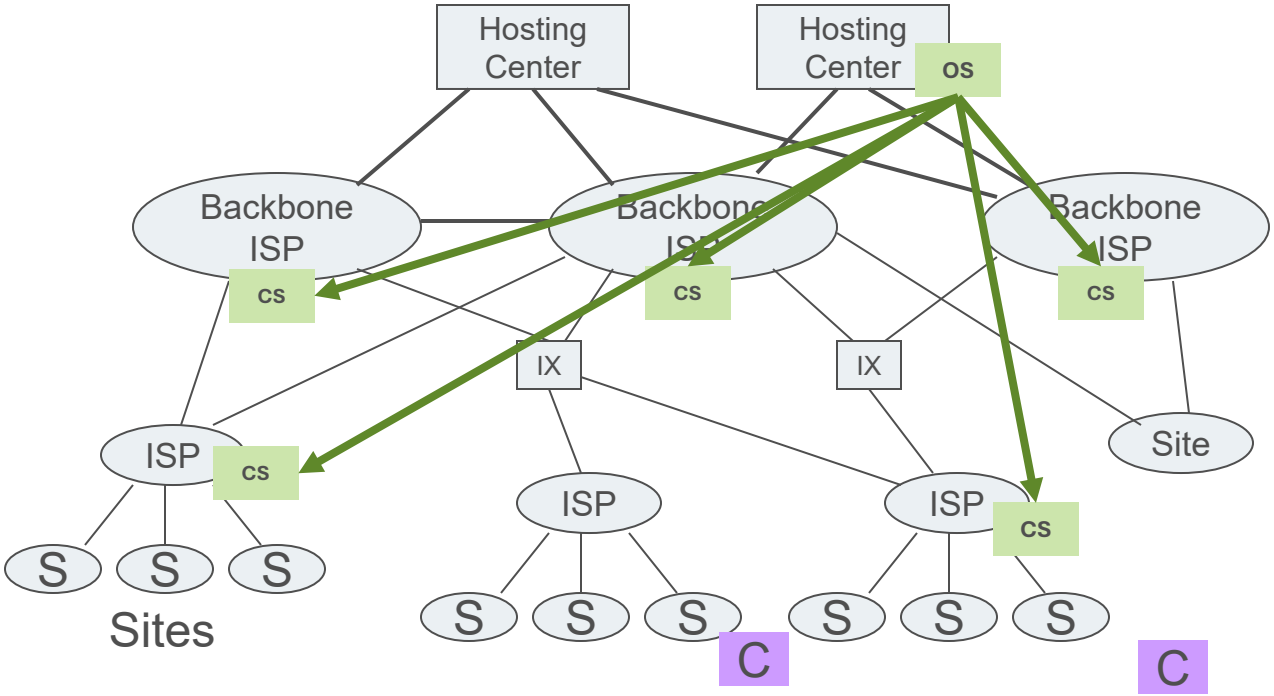
Cached CDN



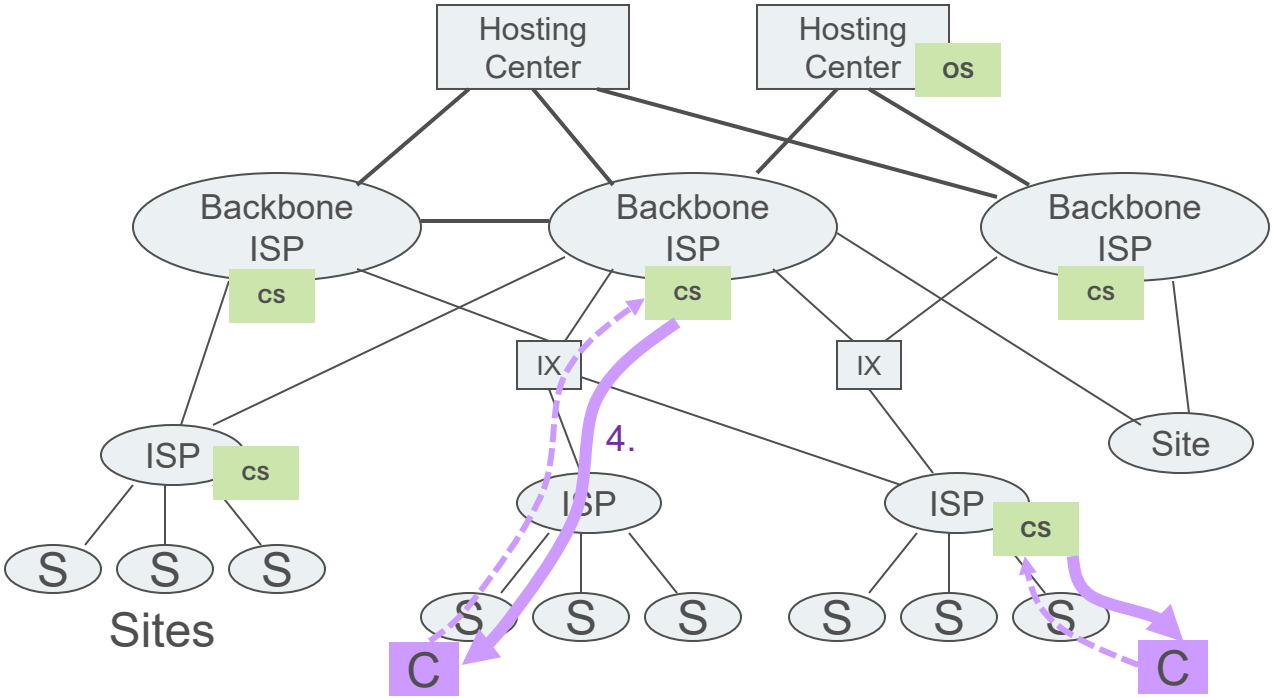
1. Client requests content.
2. CS checks cache, if content is missing it gets it from origin server.
3. CS caches content, and delivers it to client.
4. On subsequent requests, CS delivers content out of cache.

Pushed CDN

1. Origin Server pushes content out to all CSs.



Cached CDN



1. Origin Server pushes content out to all CSs.
2. Request served from closest CSs.

Summary of today's lecture

- Data-centric consistency: Sequential and causal consistency
- Grouping operations: The role of locks
- Client-centric consistency: eventual consistency and consistency guarantees (monotonic reads, monotonic writes, read your writes, and write follows reads)

- Replication management:
  - Permanent replicas, server-initiated replicas, client-initated repliacs
  - Replication in the web (Proxy-servers)

- Content Delivery Networks (CDNs) as an example of extensive replication systems in the Internet.

Suggested Reading

- Section 7.1: Introduction
- Section 7.2: Data-centric consistency models
- Section 7.3: Client-centric consistency models (7.3.1-7.3.4)
- Section 7.4.2: Content replication and placement
- Section 7.6: Example: Caching and replication in the Web

CHALMERS UNIVERSITY OF TECHNOLOGY | UNIVERSITY OF GOTHENBURG

Extra material: Replication placement

Replica(-server) placement

Figure out what the best K places are out of N possible locations.

- Select best location for which the average distance to clients is minimal. Then choose the next best server.
- Select the K-th largest autonomous system and place a server at the best-connected host. Computationally expensive.

Problem™: **Computationally expensive.**

- Position nodes in a d-dimensional geometric space, where **distance reflects latency**. Identify the K regions with highest density and place a server in all K regions. **Computationally cheap.**
- And many more possible heuristics (see Sahoo et al. 2017).

CHALMERS UNIVERSITY OF TECHNOLOGY | UNIVERSITY OF GOTHENBURG

Extra material: Content distribution

Content distribution

State, Data, or Operation? There are three basic possibilities:

- Distribute State: Propagate only a notification of an update.
- Distribute Data: Transfer data from one copy to another.
- Distribute Operation: Propagate the update operation to other copies.

Which one approach to choose depends on the operation environment and use case:

- Available bandwidth
- Read-to-write ratio at replicas

Content distribution

Pull versus Push protocols:

- Pushing updates: server-initiated approach, in which update is propagated regardless whether target asked for it.
- Pulling updates: client-initiated approach, in which client requests to be updated.

Problem™	Pushing updates	Pulling updates
1: Keeping track of client's state at server	List of client caches	None
2: Number of messages to be exchanged	Update (and possibly fetch update)	Poll and update
3: *Response time at the client*	Immediate (or fetch-update time)	Fetch-update time

CHALMERS UNIVERSITY OF TECHNOLOGY | UNIVERSITY OF GOTHENBURG

Extra material: Leases

Leases

We can dynamically switch between pulling and pushing using leases:

Definition of lease:

A contract in which the server promises to push updates to the client until the lease expires.

Lease expiration is normally time adaptive:

- **Age-based leases:** An object that has not changed for a long time, will not change in the near future, so provide a long-lasting lease.
- **Renewal-frequency based leases:** The more often a client requests a specific object, the longer the expiration time for that client (for that object) will be
- **State-based leases:** The more loaded a server is, the shorter the expiration times become

Extra material: Implementing client-centric consistency

Implementing client-centric consistency

Each write operation W is assigned a globally unique identifier by its origin server. For each client, **we keep track of two sets of writes**:

- **Read set:** the (identifiers of the) writes relevant for that client's read operations
- **Write set:** the (identifiers of the) client's write operations.

Monotonic-read consistency: When client C wants to read at server S, C passes its read set. S can pull in any updates before executing the read operation, after which the read set is updated.

Monotonic-write consistency: When client C wants to write at server S, C passes its write set. S can pull in any updates, executes them in the correct order, and then executes the write operation, after which the write set is updated.

Implementing client-centric consistency

Each write operation W is assigned a globally unique identifier by its origin server. For each client, **we keep track of two sets of writes**:

- **Read set:** the (identifiers of the) writes relevant for that client's read operations
- **Write set:** the (identifiers of the) client's write operations.

Read-your-writes consistency: When client C wants to read at server S, C passes its write set. S can pull in any updates before executing the read operation, after which the read set is updated.

Write-follows-reads consistency: When client C wants to write at server S, C passes its read set. S can pull in any updates, executes them in the correct order, and then executes the write operation, after which the write set is updated.